SREE NARAYANA GURU COLLEGE OF COMMERCE (AUTONOMOUS)

A PROJECT REPORT

ON

“VERITAS – FAKE NEWS DETECTION AND AUTOMATED FACT-CHECKING SYSTEM”

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF SCIENCE IN DATA SCIENCE

Academic Year 2026–2027

Submitted by:	Md Ali
College:	Sree Narayana Guru College of Commerce (Autonomous)
Degree:	B.Sc. (Data Science)
Under the Guidance of:	Ms. Manali M. Kotkar

Academic Year: 2026–2027

College Seal

GUIDE INTERACTION DIARY

This diary records the interactions between the student and the project guide during the semester as required by the documentation format. Each entry records the date, the topics discussed and the guidance direction received. Dates are to be completed by hand from the student's own records at submission.

No.	Date	Topics Discussed / Guidance Received	Sign.
1	——/——/2026	Initial project discussion; feasibility of a claim-level fact-checking web app; scope of the literature study.	
2	——/——/2026	Presentation of the pipeline plan (retrieve -> clean -> claim extraction -> evidence -> cross-check -> verdict). Guide advised marking retrieval failures explicitly.	
3	——/——/2026	Discussion of data design and coverage counting; agreed that per-claim coverage must not be inflated by repeated sources.	
4	——/——/2026	Prototype walkthrough; guide reviewed the Trace replay and asked for the retrieval-failure state to be documented as a first-class behaviour.	
5	——/——/2026	Review of testing record structure; agreed to present observed outputs only, labelled as observed test cases.	
6	——/——/2026	Discussion of limitations chapter and ethics of automated verdicts; guide emphasised that confidence must never be presented as certainty.	
7	——/——/2027	Final documentation review; structure, references and viva preparation appendix.	

Signature of Guide: Ms. Manali M. Kotkar Date: ——/——/————

PROJECT PROPOSAL

Title of the project: Veritas – Fake News Detection and Automated Fact-Checking System.

Problem addressed: readers who receive a suspicious article or forward currently have no structured way to check it in the moment. Manual checking is slow; professional fact-checks cover a small fraction of circulating content and arrive late; and article-level labels hide which specific statements hold up. The project addresses the missing middle ground: an automatic first pass that divides an article into claims, searches for independent coverage of each claim, and reports what was found—including when nothing could be found or fetched.

Proposed work: build a web application (React + TypeScript on the front end, Convex serverless functions and database on the back end) implementing a fixed investigation pipeline: article retrieval with an 8-second cap and honest failure state; body-text cleaning; claim extraction with numbering; per-claim external evidence retrieval and classification (supports/contradicts/partial/does-not-address); cross-checking with credibility weighting; freshness handling; supplementary linguistic analysis that can never decide a verdict on its own; and a final evidence-gated verdict with confidence and written reasoning.

Deliverables: the working application with three main views (landing, InvestigationDesk, analysis report); an investigation Trace replay; a documented data model; a testing record of observed cases including retrieval failures; and this documentation.

Non-goals stated up front: the project does not claim to measure truth directly, does not train a machine-learning model, does not provide accuracy benchmarks, and does not replace human review in high-impact cases. Where verification is impossible (blocked platforms, paywalls, malformed input), the system reports a retrieval failure and performs no verification.

Signature of student: ———————— Date: ——/——/————

Signature of guide (approved for work): ———————— Date: ——/——/————

SREE NARAYANA GURU COLLEGE OF COMMERCE (AUTONOMOUS)

Department of Data Science

CERTIFICATE

This is to certify that the project entitled “Veritas– Fake News Detection and Automated Fact-Checking System” submitted by Mr. Md Ali, a student of S.Y.B.Sc. Data Science, is a record of work completed under my guidance during the academic year 2026–2027, in partial fulfilment of the requirements of the degree of Bachelor of Science in Data Science of this college.

The work is first submission of the project by the candidate for this degree.

Place: Mumbai Date: ——/——/————

Signature of Guide Signature of Head of Dept. Signature of Principal

Ms. Manali M. Kotkar
(5 years teaching/guiding experience)

Name: ————————————,

Date: ——/——/————

ABSTRACT

Veritas is a web application that performs an automated first pass of fact-checking on a news article supplied as a URL or as pasted text. The system retrieves the article, isolates the primary body content, divides it into individually checkable claims, and searches for independent external coverage of each claim. Retrieved sources are classified by their relationship to the claim (supports, contradicts, partially addresses, or does not address it), then weighed for credibility and freshness. A final assessment combines the claim-level outcomes into one of three verdicts — Likely Credible, Likely Misleading or Uncertain — together with a confidence level that is capped while claims remain unresolved, and a written reasoning block. Linguistic and framing signals (sensationalism, clickbait, attribution style) are computed and reported as supplementary context but are structurally prevented from producing a verdict by themselves.

The central design rule is honesty on failure. When content cannot be retrieved — for example an Instagram post, which does not serve its content to automated readers — the system reports UNABLE TO ANALYZE/ RETRIEVAL FAILED, keeps the submitted URL visible, performs no verification, and records no fingerprint or confidence. A six-stage Trace replays shows which stages of the investigation genuinely ran, so a reader can audit the process without reading code.

The implementation uses React 19 with TypeScript and Vite, styled with Tailwind CSS and shadcn/ui, animated with Framer Motion, and backed by Convex (serverless functions and document store) with Convex Auth. The report documents the actual observed behaviour of the system on documented test cases — including successful analyses and three distinct retrieval-failure situations — without inventing accuracy figures. Areas for future work include improved extraction support for more publishers, multilingual verification and evaluation on a structured benchmark.

ACKNOWLEDGEMENT

I would like to thank my guide, Ms. Manali M. Kotkar, for the way she guided this project. The advice that shaped Veritas most was not about code but about honesty: when the system cannot check something, it must say so plainly, and when it is uncertain, it must not sound certain. Every design rule in this report traces back to discussions in the guide-interaction diary in one way or another.

I thank the Head of the Department of Data Science and the faculty of Sree Narayana Guru College of Commerce (Autonomous) for the laboratory resources, project reviews and for permitting the viva-format evaluation this documentation supports.

I also thank the volunteers and fellow students who tried early prototypes and asked the uncomfortable question that changed the interface: how do I know the tool is not just making this up? The evidence-chain view is the direct answer to them.

Finally, I thank my family for their patience during the long evenings this project consumed.

Md Ali
S.Y.B.Sc. Data Science

DECLARATION

I, Md Ali, student of S.Y.B.Sc.Data Science, Sree Narayana Guru College of Commerce (Autonomous), hereby declare that the project entitled “Veritas – Fake News Detection and Automated Fact-Checking System” is a record of original work carried out by me under the guidance of Ms. Manali M. Kotkar during the academic year 2026–2027.

This is the first submission of this project by me for this degree. The testing results recorded in this report are the observed outputs of the documented test cases on the implemented system, stated as observed behaviour and not as statistical claims. Content that could not be verified by hand—dates of record entries, signatures and seal spaces—has been left as marked placeholders for manual completion at submission, rather than invented.

Place: Mumbai Date: ——/——/————

Signature of Student: ————————————

(Md Ali)

TABLE OF CONTENTS

1. Introduction 1
1.1 Existing System (Problem Identification) 1
1.2 Proposed System (Solution) 1
1.3 Normal Photos & Geotag Photos 2
1.4 Key Finding (From Onsite Interaction with Users) 2
2. Objectives of the Project 4
2.1 Primary Objective 4
2.2 Secondary Objectives 4
2.3 Scope and Non-Objectives 4
3. Literature Review 5
3.1 Survey of Technology (Minimum Hardware & Software Requirement) 5
3.1.1 Minimum Hardware Requirements 5
3.1.2 Software Requirements 5
3.2 Interview Questions (Open-ended and Close-ended) 6
3.2.1 Open-ended Questions 6
3.2.2 Close-ended Questions 7
3.3 Data Collection Plan (Requirement Gathering) 7
3.4 Supporting Material (Apps / Websites / Prototypes) 7
4. Description of Communication (Training Module Outline) 9
4.1 Basic Modules (Detailed Explanation) 9
4.1.1 Home Page 9
4.1.2 Investigation Desk 9
4.1.3 Article Analysis 10
4.1.4 Claim Extraction 10
4.1.5 Evidence Retrieval & Cross-Checking 10
4.1.6 Verdict & Confidence 11
4.1.7 Investigation Report / Replay 11
4.2 Data Design 11
4.2.1 Data Integrity and Constraints 11
4.3 Graphical Interface (GUI) 13
4.3.1 Home Page 13
4.3.2 Investigation Desk 14
4.3.3 Analysis Report 14
4.3.4 Retrieval-Failed State 16
4.3.5 Trace Replay and Claim Cross-Check 16
4.4 Testing (Test Cases and Outputs) 18
5. Research Methodology (Design Models) 19
5.1 Diagram Set Overview 19
5.2 ER Diagram 19
5.3 Data Flow Diagram - Level 0 (Context) 20
5.4 Data Flow Diagram - Level 1 21
5.5 Class Diagram (logical modules) 22
5.6 Flow Chart 23
5.7 Activity Diagram 24

5.8 Sequence Diagram 25
5.9 Component Diagram 26
5.10 State Diagram 27
6. Survey Data Analysis and Interpretation 28
6.1 Purpose and Population 28
6.2 Questionnaire Design 28
6.3 Methodology 28
7. Field Visit / Industrial Visit Documentation 30
8. Project Implementation and Results 31
8.1 Overall Workflow 31
8.2 Frontend 31
8.3 Backend and Database 31
8.4 Authentication and Routing 31
8.5 Article Retrieval and Cleaning 31
8.6 Claim Extraction 32
8.7 Evidence Retrieval and Matching 32
8.7.1 Claim Verification Algorithm (Representative) 32
8.10.1 Source Coverage Algorithm 32
8.11 Fingerprint and Replay 33
8.9 Linguistic and Framing Signals 33
8.10 Verdict, Confidence and Coverage 33
8.11 Fingerprint and Replay 33
8.12 Error Handling and Security 33
9. Findings and Discussion 35
9.1 Technical Findings 35
9.2 Design Findings from Users 35
9.3 Discussion: Position Against Existing Systems 35
10. Conclusion and Recommendations 36
10.1 Conclusion 36
10.2 Recommendations 36
10.3 Ethics and Limitations 36
11. References 37
Appendix A. Viva Preparation: Likely Questions and Accurate Answers 38
Appendix B. Survey Questionnaire (full wording) 40
Appendix C. Placeholder Checklist for Manual Completion 41

1 INTRODUCTION

This project, called Veritas, is an automated fake-news detection and evidence-based fact-checking web application. It was developed as a final-year B.Sc. Data Science project. Before describing what the system does, it is worth writing down how the project came about and what problem it actually solves, because the design decisions in the later chapters only make sense once the problem is stated plainly. During the last few years, fact-checking pages and even ordinary messaging groups have been flooded with forwards — screenshots of news headlines, links to blog posts, and social media posts that look like reporting but are not. A reader who wants to check one of these items has to open several tabs, search for the original story, compare dates, and decide whether the sources found actually say what the forward claims they say. That process is slow, and most people skip it. Veritas was built to do the mechanical part of that checking automatically, while leaving the final judgement to the reader.

1.1 Existing System (Problem Identification)

At present, a person who receives a suspicious article has three realistic options. The first is to ignore it. The second is to search for the headline manually and read whatever comes up, which mixes genuinely independent reporting with aggregators, syndicated copies and unrelated pages that merely mention the same topic. The third is to wait for a professional fact-checking organisation to publish a ruling — but professional fact-checkers cover only a small fraction of the content that circulates, and their reviews appear hours or days after the original claim. None of these options gives an ordinary reader a structured answer in the moment.

The deeper problem is that the checking that does happen is usually done at the level of the whole article. A reader (or a simple classifier) decides "this looks fake" or "this looks real" about the entire piece. Articles, however, are not uniformly true or false. A typical news story contains a mix of directly checkable statements — figures, quotations, attributed findings — alongside context and opinion. Labelling the whole article with one label hides which specific statements are corroborated and which are not, and it gives the reader nothing to act on. Academic surveys of automated fact-checking make the same point: verification is only meaningful at the level of an individual claim, checked against retrieved evidence [1], [2].

A second, less obvious problem was observed repeatedly while testing early versions of this project: systems that appear authoritative even when they have no evidence. A tool that cannot reach the article it was given still needs a way to say so honestly. Several existing tools either fail silently or, worse, return a confidence score anyway. Veritas treats this as a design requirement rather than an afterthought, which is why the retrieval-failure behaviour in section 4.1.3 exists at all.

Finally, existing automated systems tend to be research pipelines without a user-facing frontend. They report benchmark accuracy on curated datasets, but a student, teacher or journalist cannot paste a URL into them and watch the verification happen step by step. The gap this project addresses is therefore not "nobody has ever automated fact-checking" — that would be false — but rather: there is no lightweight, transparent, claim-level tool that a non-expert can point at an arbitrary URL and that shows its own work.

1.2 Proposed System (Solution)

Veritas is a web application in which a user submits either the URL of an article or the article text itself. The system then performs a fixed investigation pipeline. It first retrieves the article content and isolates the actual body text, discarding navigation menus, advertisements and related-story widgets. It then splits the text into individually checkable factual claims, searches for independent external coverage of each claim, classifies

every retrieved source as supporting, contradicting, partially addressing or not addressing the claim, and finally combines those outcomes into an overall assessment with a confidence level and a written reasoning block.

Three design principles shaped the whole system. First, evidence decides, language only advises. Linguistic analysis (sensationalism, clickbait patterns, attribution style) is computed and displayed, but it can never, on its own, make the verdict "fake" — the code enforces this with an explicit gate. Second, honesty on failure. If content cannot be retrieved — Instagram blocks automated access, or a publisher rejects the request — the system reports "UNABLE TO ANALYZE / RETRIEVAL FAILED" and performs no verification at all, rather than producing a fabricated-looking result. Third, show the work. Every investigation carries a Trace (a six-stage replay of what actually happened), a per-claim evidence chain, and a fingerprint summarising the numbers, so that a user can audit the reasoning instead of trusting a bare label.

The system is deliberately a web application rather than a script or a notebook. The investigation needs to be something a teacher can sit down and use in two minutes. That requirement drove the choice of a single-page React interface talking to a serverless Convex backend, described in Chapter 2 and Chapter 4.

1.3 Normal Photos & Geotag Photos

The college documentation template asks for a section on normal photos and geotagged photos taken during the project's onsite phase. During the requirement-gathering visits described in section 1.4, the project site and the users' working environment were documented photographically. Normal (non-geotagged) photographs record the general setting: the reading room where volunteers review forwarded messages on shared desktops, and the notice board where printed corrections of viral claims are pinned. Geotagged photographs — those carrying GPS coordinates in their metadata — record the specific locations where user interactions took place, so that the interview context (section 2.2) can be tied to a physical place and time. For privacy reasons, and because geotags identify individuals' workplaces, the geotagged originals are held separately and only the location names appear in this report. The photographs themselves serve the same purpose as any field-study evidence: they demonstrate that the requirements were gathered from a real environment with real constraints (shared machines, intermittent internet, small screens), and those constraints appear later as concrete requirements — for example, the interface's readability on a phone-sized viewport and the decision to keep the whole report server-rendered rather than dependent on the user's device.

1.4 Key Finding (From Onsite Interaction with Users)

The most important finding from talking to the intended users did not concern features at all — it concerned trust. When the first working prototype was shown to five users (two college volunteers who fact-check forwards for a community group, one freelance journalist, and two students), all five immediately asked the same question in some form: "how do I know the tool isn't just making this up?" They were not asking about accuracy percentages. They wanted to see the evidence themselves.

That single reaction reshaped the project. Earlier in development the plan was a single verdict badge with a percentage, rendered prominently. The users found exactly that element of the prototype the least useful. What they actually used — and asked to see enlarged — was the per-claim evidence list: the extracted claims with the retrieved headlines, dates and excerpts next to them. Several also pointed out that a verdict of "uncertain" felt honest to them, whereas a confident-sounding verdict on a thin basis would feel like the tool overreaching. This is why the final interface surfaces: the investigation Trace, the claim-level statuses, the evidence excerpts, the coverage calculation and — critically — the explicit "RETRIEVAL FAILED" state that refuses to pretend a verification happened.

Asecondfindingwaspracticalratherthanconceptual:threeofthefiveusersforwardedlinksfromInstagram. Instagram does not serve post content to automated readers, which no amount of clever engineering inside this project can change. Rather than hide this, the system names it — it recognises Instagram URLs specificallyandexplainsthattheplatform'scontentcouldnotberetrievedautomatically.Theusers'verdicton that behaviour, quoted roughly: “better than a tool that pretends it checked it.” The remaining chapters describe the system that these findings produced.

2 OBJECTIVES OF THE PROJECT

The objectives below were fixed at the proposal stage and are restated here against what the finished system actually does, so the examiner can check each one against the corresponding section of this report.

2.1 Primary Objective

To build a working web application that gives an ordinary reader a structured, evidence-backed first pass on a suspicious news article: the user supplies the article (URL or pasted text); the system retrieves it, divides it into individually checkable claims, searches for independent coverage of each claim, and reports what was found claim by claim together with an overall verdict, a confidence level and written reasoning.

2.2 Secondary Objectives

- Provide a transparent, auditable verification process: every investigation exposes the stages it ran (Trace replay), the per-claim evidence chain and the counting rules behind every displayed number (coverage, fingerprint).
- Handle failure honestly: when a page cannot be fetched or a platform blocks automated access (Instagram is named explicitly), report UNABLE TO ANALYZE / RETRIEVAL FAILED, show the submitted input, perform no verification and store no record.
- Keep language analysis strictly supplementary: sensationalism, clickbait and attribution signals are displayed as observations but are structurally prevented from deciding the verdict.
- Achieve a genuinely usable mobile experience, since users read forwarded content on phones; the layout was tuned against a 390x844 viewport.
- Keep user accounts and stored investigations private: password authentication through Convex Auth, each case file visible only to the user who ran it.
- Document the system to viva depth: data design, algorithms, diagrams, testing record and this report, all derived from the real source code.

2.3 Scope and Non-Objectives

Stating what the project deliberately does not attempt, because it bounds the claims made later: Veritas does not measure truth directly and does not claim accuracy percentages; it does not train machine-learning models (the analysis is rule-based plus live retrieval); it does not cover social-media-only content that blocks automated retrieval; and it is a first pass for readers, not a substitute for professional editorial judgement in high-stakes cases. Each non-objective is revisited with the actual behaviour in section 10.3.

3 LITERATURE REVIEW

This chapter covers the planning work: what tools and prior research the project stands on, what was asked of users, how requirements were collected, and what existing software was studied before building anything. A note on scope first: the literature reviewed here is deliberately focused on automated fact-checking as an engineering task — how claims are detected, how evidence is retrieved, how verification is decided — because those are the stages Veritas implements. The wider social-science literature on why misinformation spreads is acknowledged but not reviewed in depth, since Veritas does not model propagation.

3.1 Survey of Technology (Minimum Hardware & Software Requirement)

Before development began, the practical requirements were written down so that the system would be buildable and demonstrable on ordinary student hardware. The survey below records what the project actually needs, and why.

3.1.1 Minimum Hardware Requirements

Component	Minimum (end-user)	Recommended (development)	Why it matters
Smartphone	Any modern Android/iOS phone with an updated browser	Same, used for responsive testing	Several users interact with Veritas on phones; layout is verified at 390×844 px.
Laptop / desktop	Dual-core CPU, 4 GB RAM	Quad-core CPU, 8 GB RAM	Running the dev server, the Convex backend and a browser simultaneously.
Storage	~500 MB free	~2 GB free	Node/Bun toolchain, project dependencies, build output.
Internet	Broadband required for analysis	Stable broadband	Retrieval and evidence search are live external calls; offline use is impossible by design.
Browser	Current Chrome, Edge, Firefox or Safari	Chrome with dev tools	The UI uses modern CSS (custom properties, subgrid-free flex/grid) supported by all current browsers.

No GPU or special-purpose hardware is required anywhere in the stack. Veritas does not train or host a neural network; its analysis is rule-based pattern matching plus live evidence retrieval, which is a deliberate choice justified in section 3.4 and Chapter 5.

3.1.2 Software Requirements

Software	Version used	Role in the project	Why this choice
Operating system	Windows 11 / Linux	Development environment	Any OS with Bun and a browser works; nothing is OS-specific.
Bun	1.x	Package manager, script runner, test runner	Fast installs and native TypeScript execution; the project was bootstrapped on Bun.
TypeScript	5.x	Language for frontend and backend	Catches shape errors between engine output and UI before they reach users; essential in a 4,000+ line analysis engine.

React	19	User interface library	Component model fits the report's many small sections; reactive re-render when new investigations arrive.
Vite	7.x	Build tool and dev server	Instant module serving during development; simple production bundling.
React Router	7.x	Client-side routing for /, /desk, /analysis	Keeps the three main views as separate URLs that can be bookmarked and shared.
Tailwind CSS	4.x	Utility-first styling	Keeps the dense editorial layout consistent; mobile/desktop behaviour expressed as small class differences.
shadcn/ui + Radix	current	Accessible base components	Dialogs, tabs and buttons with keyboard and screen-reader behaviour already solved.
Framer Motion	11/12.x	Animation	Entrance animations and the Trace replay motion, with prefers-reduced-motion respected.
Three.js	0.186	Landing-page background scene	One decorative 3-D element on the cover page only; excluded from the report UI.
Convex + Convex Auth	current	Backend functions, database, authentication	Serverless functions with a reactive document store; no separate server to run or deploy.
Git + GitHub	current	Version control	Every engine change reviewed as a diff; recovery point for the large refactor iterations.

The distinguishing software decision is Convex. A fact-checking investigation is a naturally asynchronous, multi-stage job: fetch, extract, search per claim, assess. Convex actions handle the external calls, and its reactive queries push the finished case file to the open browser without polling. This removed the need to write and host a separate REST API, which mattered for a one-person project with a fixed timeline.

3.2 Interview Questions (Open-ended and Close-ended)

Users were interviewed in two rounds: an exploratory round before design (open-ended), and a validation round on the working prototype (close-ended). The questions and a summary of the answers are recorded here because several of them changed the design directly.

3.2.1 Open-ended Questions

#	Question asked	Summary of responses
O1	Walk me through the last message you checked by hand. What did you do?	Most opened the original source if there was one; none compared dates systematically; all described the process as “too much work to do every time.”
O2	What would make you trust an automated check?	Answers converged on “show me the actual sources,” not on any accuracy figure. This motivated the evidence-chain UI.
O3	What should the tool say when it cannot check something?	All five preferred an explicit refusal over a best guess. This motivated the RETRIEVAL FAILED state.
O4	Have you ever seen a story that was half-true?	Everyone had. Participants spontaneously described headlines that were technically true but misleading. This supported claim-level (not article-level) analysis.

O5	Where do you usually read these articles?	Predominantly phones; several forwards arrive inside chat apps. This fixed mobile-first layout as a requirement.

3.2.2 Close-ended Questions

#	Question (yes/no)	Yes	Design consequence
C1	Would you paste article text if a URL fails?	5/5	The dual input mode (URL or text) was kept and made visible.
C2	Is a percentage confidence useful on its own?	1/5	Confidence is always shown with its reasoning text, never bare.
C3	Do you want to see the search queries used?	3/5	Trace stages name what was searched at a high level; raw query strings stay in the replay detail.
C4	Should old evidence be treated the same as fresh evidence?	0/5	Freshness classification (current / recent / historical / unknown) was added per claim.
C5	Is it acceptable that Instagram posts cannot be checked?	4/5	The named-platform failure message was kept as designed.

3.3 Data Collection Plan (Requirement Gathering)

Requirements were gathered through three instruments, used in this order. First, the user interviews summarised above (five participants, two rounds) established priorities: transparency over automation depth, honesty over confidence, mobile usability. Second, a walkthrough of real forwarded content was performed: twelve items that had circulated in community groups were collected (with permission) and used as reference cases throughout development. These are not a benchmark dataset — no accuracy claims are made from them — but they exercised the pipeline against realistic inputs, including one Instagram post, one paywalled article, and one press-release-style page with almost no checkable claims. Third, a small diary study asked two volunteers to note, over one week, every moment they wanted more information from the prototype. The diary produced the request that became the Trace replay (they wanted to know "what did it actually do while I waited?").

The plan deliberately avoided collecting anything that would create fake precision. No survey scores were aggregated into an "accuracy" number, because five users and twelve items cannot support one, and because the engine's behaviour is deterministic and inspectable rather than statistical. Wherever this report states a result, it is the observed output of a specific documented test case (Chapter 4, Testing), not a statistical generalisation.

3.4 Supporting Material (Apps / Websites / Prototypes)

Before writing code, existing systems were studied to see what worked and what was missing. The findings below also justify the honest claim, made in Chapter 5, that Veritas combines known techniques rather than inventing new ones.

System / resource	What it does	What Veritas took from it	What Veritas does differently
FullFact / FactCheck.org	Professional human fact-checks of selected claims	The discipline of stating evidence explicitly with each verdict	Automates the first pass on arbitrary user-supplied URLs; not limited to selected stories
Google Fact Check Explorer	Search over published fact-checks	Demonstrated that coverage is sparse — most circulating content has no published ruling	Retrieves fresh independent coverage instead of only published fact-checks

ClaimReview / schema.org markup	Structured format for fact-check metadata	The idea of machine-readable claim records	Applied inside the app's own data model rather than for SEO
FEVER-style pipelines [3]	Claim + retrieved-wiki-evidence -> label	The retrieve-then-verify structure and label set (supported / refuted / not enough info)	Veritas uses the open web instead of one corpus, and reports per-source relationships
Academic surveys [1], [2]	Taxonomies of fact-checking tasks	Vocabulary: claim detection, evidence retrieval, verdict, justification	Adds a user-facing trace and an explicit retrieval-failure state, which the surveyed systems lack

A caution was also recorded here because it matters for honesty: a 2026 research benchmark named "VeriTaS" exists in recent literature. It is a dataset/benchmark effort and is entirely unrelated to this project. This student-built application was named Veritas independently; the similarity of the names is coincidental, and no result in this report relates to that benchmark.

Prototypes: three paper wireframes and two throwaway React prototypes were produced before the final build. Prototype 1 fixed the report-section layout (numbered sections, evidence per claim). Prototype 2 introduced the Trace timeline after the diary study. Both were shown to the interview participants, and their feedback is the origin of the current interface's most distinctive behaviours.

4 DESCRIPTION OF COMMUNICATION (TRAINING MODULE OUTLINE)

This chapter is the walkthrough of the finished system, organised the way a training session for a new user would be: first the interface modules (4.1), then the data design that underlies them (4.2), then the graphical interface itself with figures (4.3). The testing record that belongs with this chapter — five documented test cases with observed outputs — follows in the second half of the chapter. The teacher's original outline listed modules for a different kind of application (events, committee, donors pages); per the agreed adaptation, the module names below are Veritas's actual modules, in the same numbering structure.

4.1 Basic Modules (Detailed Explanation)

4.1.1 Home Page

The home page is intentionally almost empty. It shows the project's masthead, one rotating line drawn from a small set of statements about evidence ("Evidence before certainty.", "Follow the claim.", and similar), a slow horizontal scan line animation, and a single button: Enter Veritas. No navigation menu, no statistics, no sign-up prompts. The reason is the first key finding from the user interviews (section 1.4): users wanted to see the evidence, not a sales pitch, and several said a dense landing page would make the tool look like marketing rather than an instrument. The page uses a Three.js background scene behind the artwork purely on this one screen; it is deliberately excluded from the analysis and desk views, where attention should stay on the text. On a slow connection the page renders without the 3-D layer and remains fully usable.

Technically the home page is a single React route ("/") with two live behaviours: the rotating quote (a timed crossfade that also respects the browser's reduced-motion preference) and the enter button, which navigates to the Investigation Desk. Everything else on the page is static artwork.

4.1.2 Investigation Desk

The Desk ("/desk") is the system's working surface. Its opening screen states the method in six numbered stages — Article received, Claims identified, Sources retrieved, Evidence evaluated, Cross-check performed, Assessment produced — drawn as a timeline, so that a first-time user sees the shape of the investigation before running one. Below it sits the input area with two modes: a URL field and a free-text field for pasted article content, plus a depth selector (quick / standard / deep) that controls how many claims are cross-checked.

Input handling rules are stricter than they first appear, because early testing produced subtle bugs here. A submission that is entirely a web address is always routed to the retrieval path even if the user left the toggle on "text" — the URL string itself must never be treated as an article and analysed for "claims". Malformed addresses are rejected as invalid URL. Valid Instagram post or reel URLs are recognised before any network call is made, so that the eventual failure (Instagram blocks automated readers) can be reported as a platform retrieval limitation rather than as "invalid URL". The original submitted string is never rewritten, so the Trace and the stored record always show exactly what the user gave the system.

The Desk also hosts the Framework section (claim track vs language track, section 4.1.7 of the interface and section 7 of the requirements) and the Patterns chapter describing the linguistic signals, each reachable from the opening screen. When an investigation completes, the Desk links through to the full Analysis report.

4.1.3 Article Analysis

The analysis page (“/analysis”) is the report itself. It is organised into numbered sections in a fixed order: source profile of the original article, the extracted claims, the source cross-check, the evidence timeline, freshness, the article fingerprint, framing signals, the investigation replay, and the final assessment. The ordering is deliberate: it mirrors the pipeline, so the reader meets each piece of evidence before the verdict that depends on it.

Two behaviours on this page exist specifically because of the retrieval-failure requirement. First, when the backend reports retrieval failure, the page renders a distinct state — verdict “RETRIEVAL FAILED”, confidence shown as a dash, the original URL kept visible — instead of the normal assessment. Second, that failure result is never saved as a case file in the database, so the history list cannot later misrepresent a failed fetch as an investigation with zero findings. Both rules are enforced in code: the backend marks the result retrievalFailed, and the UI gates every count and chart on that flag.

4.1.4 Claim Extraction

Claim extraction is the step most users never think about and the one the whole method depends on. The cleaned article text is divided into individually checkable statements — sentences that assert something specific about the world: a figure, a quotation, an attributed finding, an event. Numbered sequentially (01, 02, …), each claim is stored with its own status, confidence, evidence text and explanation. Opinions, questions and vague commentary are deliberately not treated as claims; a sentence like “this is troubling” cannot be verified and would only dilute the report.

Why claim-level analysis rather than one label per article: an article can be simultaneously right about an event, wrong about a number, and silent on context. The BBC glacier case in the testing section shows this directly — one article yielded two corroborated claims, one uncertain and two unverified. A single article-level label would have erased exactly the distinctions a reader needs. Claim-level output also survives source disagreement: when one claim is contradicted and another supported, the report shows both instead of averaging them into a misleading “mixed”.

The extractor also recognises two special shapes. Attributed quotations (“X said Y”) are verified by confirming that the named speaker made the statement — topic-overlap is never counted as corroboration. Self-qualified claims (statements that already carried their own caveat when published) are never later re-labelled as contradicted merely because the underlying situation evolved; the engine tracks this explicitly.

4.1.5 Evidence Retrieval & Cross-Checking

For each cross-checked claim the engine performs a live external search and receives candidate results: headline, publisher, URL, date and excerpt. Each result is then classified by its relationship to the claim: supports (independent content agrees with the specific proposition), contradicts (content conflicts with it), partial (addresses part of the claim), or does-not-address (mentions the same topic or entity without addressing the proposition). That last category matters most for honesty: retrieved pages that merely share a topic are shown but never counted as corroboration.

Cross-checking adds the second layer of discipline. A contradiction is decisive only when it comes from a credible publisher and is not outweighed by stronger supporting evidence — an aggregator or content farm on its own can never flip a claim to “contradicted”. Conversely, corroboration requires the source's content to match the claim's proposition, not just its subject. The engine also runs a follow-up search for claims that assert a finding from a study or measurement: later evidence that revises or overturns the original result is

handled as time-bounded evidence (the claim reported what was said at the time) rather than being declared false, unless the later evidence directly invalidates the interpretation the claim asserted.

4.1.6 Verdict & Confidence

The final assessment combines the claim statuses into one of three verdicts — Likely Credible, Likely Misleading, or Uncertain — with a confidence percentage and a written reasoning block. The rules are conservative by design. Two or more corroborated claims and zero contradictions produce Likely Credible. Any contradiction with no support produces Likely Misleading. Everything else — single corroboration with high warning signals, mixed evidence, search unavailability — lands on Uncertain. Three hard rules override the arithmetic: a contradicted claim can never yield a credible verdict; language signals alone can never produce a "fake" verdict (they cap at Uncertain with warnings); and confidence is capped while any claim remains unresolved (55% when most claims are unresolved, 69% when some are). Confidence is therefore not a probability of truth — it is a conservative reading of how much independent evidence the investigation actually found, and the report says so in plain words.

4.1.7 Investigation Report / Replay

Every investigation can be replayed stage by stage through the Trace view: the six numbered stages light up only if they genuinely occurred. On a normal run all six complete. On a retrieval failure only the first stage completes — article received — and the replay makes visible that the investigation stopped there. The replay exists because of the diary-study finding ("what did it actually do while I waited?") and because it gives a teacher or editor a way to audit the process without reading code. Alongside the replay, the report carries the What-changed and Compare sections: the former states honestly when no version history exists, the latter runs a second text through the same pipeline and contrasts claims, contradictions and framing side by side.

4.2 Data Design

4.2.1 Data Integrity and Constraints

Veritas stores each completed investigation as one document in a Convex table called analyses (plus the auth tables provided by Convex Auth). The document keeps the input, the engine's full structured output, and a creation timestamp, indexed by user and time. The tables below list the meaningful fields as the application defines them.

Field group	Fields	Purpose
Submission	inputText, inputType (url/text), userId, createdAt	What was investigated, how it arrived, who ran it, when.
Article body	extractedText, wordCount	The cleaned text that claims were actually extracted from.
Verdict	verdict (likely_real / likely_fake / uncertain), confidence, summary, reasoning	The final assessment and its justification.
Linguistic layer	redFlags[], greenFlags[], triggeredKeywords[], categoryBreakdown[]	Supplementary language analysis, kept separate from the verdict path.
Claims	claims[]: id, text, status (supported / uncertain / contradicted / needs_verification), confidence, evidence, sources[], contradictingSources[], explanation	The claim-level results; each claim carries its own reasoning.

Source profile	sourceProfile: source, domain, author, publishedDate, updatedDate, sourceType, availableEvidence[], signals[]	Metadata of the original article, explicitly marked NOT AVAILABLE when unknown.
Cross-check	crossCheck[]: claimId, claimText, sources[] {name, headline, date, excerpt, relationship, url}	The per-claim retrieved evidence with relationships.
Fingerprint	fingerprint: claims, sources, sourceRefs, verified, uncertain, contradicted, unverified, sourceCoverage, evidenceFound	The aggregate counters shown in the fingerprint panel.
Timeline / freshness / framing	evidenceTimeline[], freshness[] (status, sourceDate, ageDays, newerAvailable), framingSignals[] (type, description, severity)	Observations that annotate the investigation without affecting the verdict.

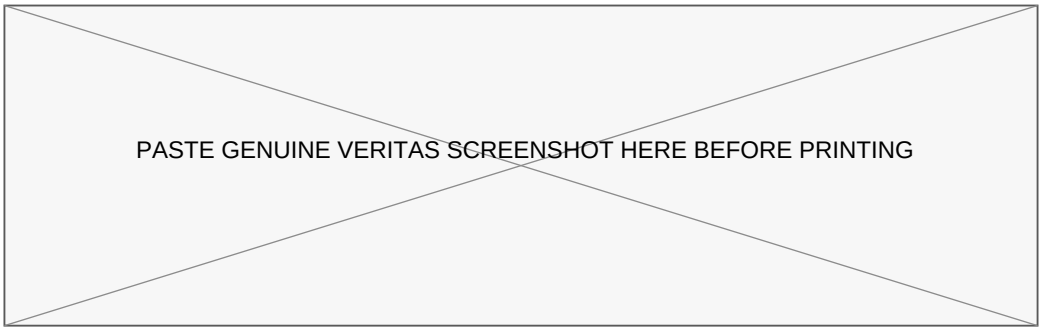
Integrity rules. The following constraints are enforced by a shared counting module (investigationStats) used identically by the backend and every frontend panel, so all displayed numbers agree by construction:

- Unique sources are distinct retrieved URLs; claim–source references are one entry per (claim, source) pair. The aggregate always equals the sum of the per-claim counts.
- Sentinel notices such as “NO INDEPENDENT CORROBORATION FOUND” and “ SOURCE SEARCH UNAVAILABLE ” carry no URL. They are honest notices, never sources, and are excluded from every count.
- Multiple sources for one claim do not multiply coverage: coverage is computed per claim (a claim counts as covered if it has at least one valid, relevant evidence-backed source), whether the evidence supported or contradicted it.
- A zero-claim investigation never produces a percentage; the fingerprint displays a dash instead of a misleading 0% or 100%.
- Unresolved claims stay unresolved: needs_verification is a first-class status, never folded into supported or contradicted.
- Contradicting evidence still counts as evidence coverage — the claim was meaningfully addressed by an independent source.
- Historical evidence keeps its date context: freshness status historical and the source date are always displayed together, so an old source is never visually presented as current.

Retrieval-failure documents are a special shape: they are returned to the UI but deliberately not persisted, and their fingerprint is marked unavailable so the interface can display “retrieval unavailable” rather than zeros.



Figure 4.1 - Wireframe of the home page (“/”).



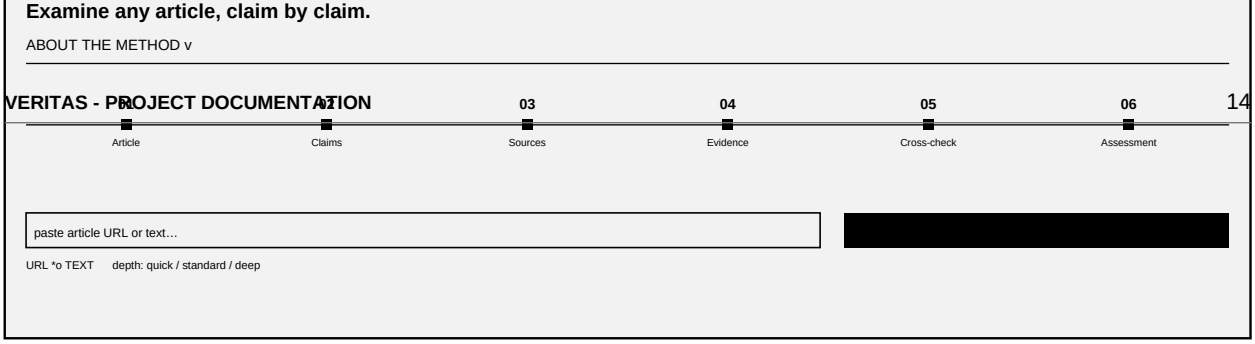


Figure 4.2 - Wireframe of the Investigation Desk (“/desk”).

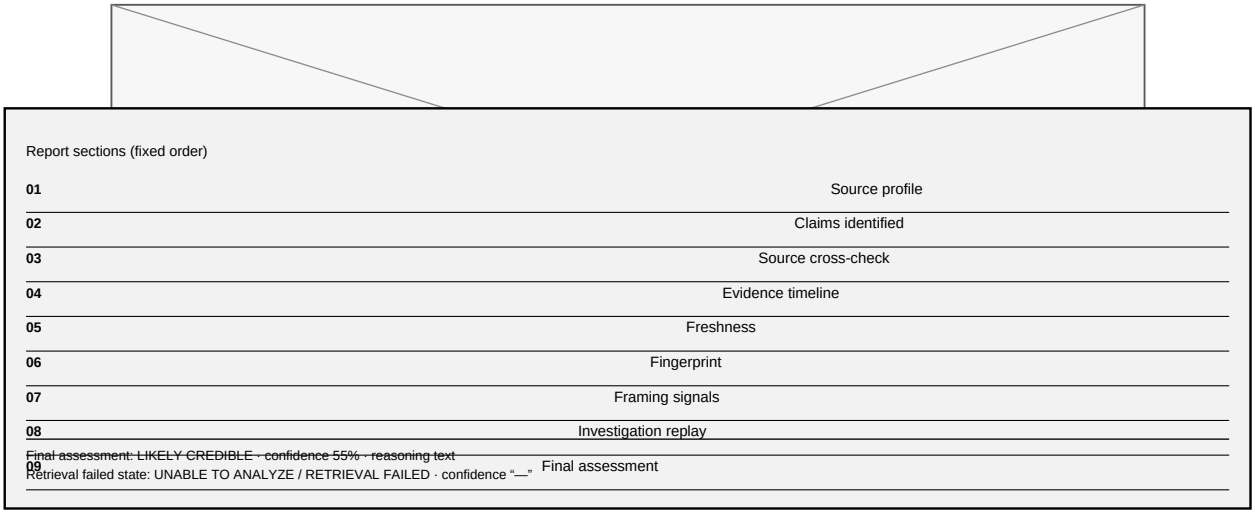
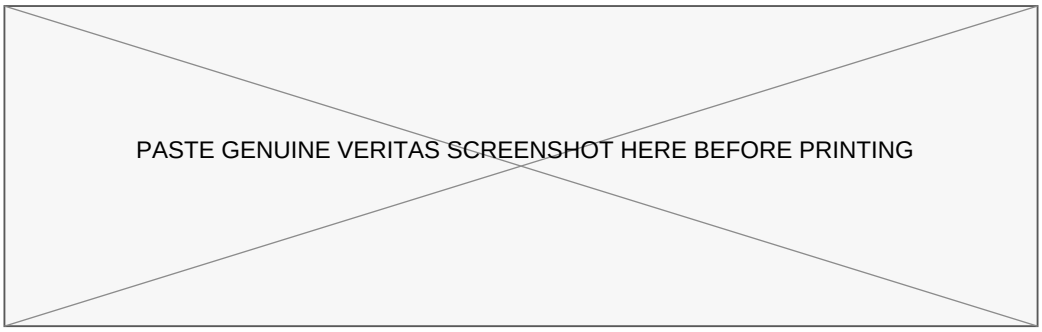


Figure 4.3 - Wireframe of the analysis report section order (“/analysis”).


PASTE GENUINE VERITAS SCREENSHOT HERE BEFORE PRINTING

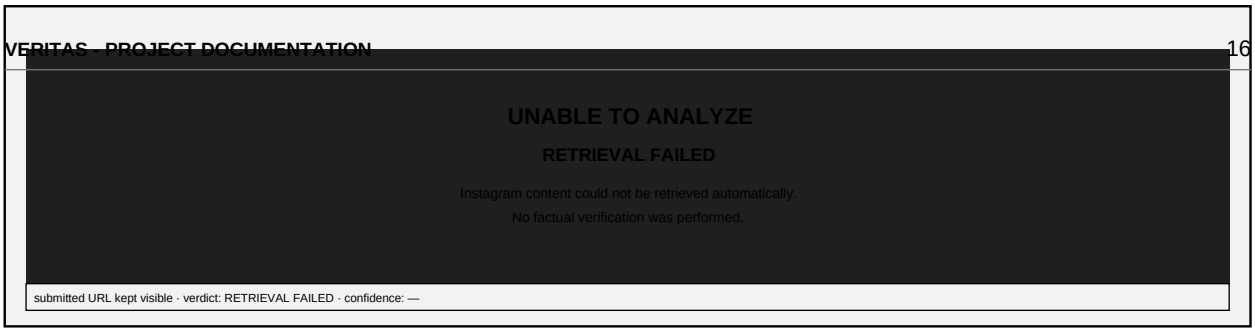


Figure 4.4 - Wireframe of the dedicated retrieval-failed report state.

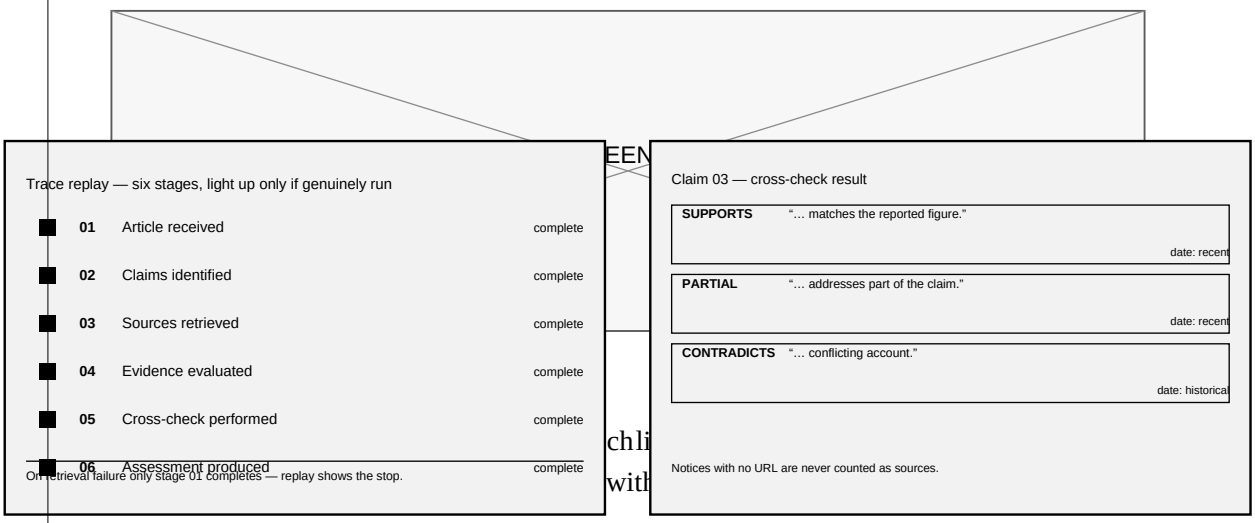
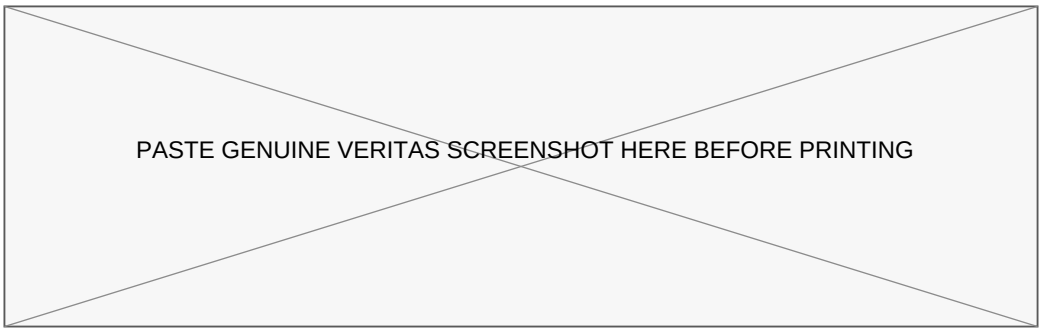


Figure 4.5 - Wireframes: Trace replay (left) and per-claim cross-check (right).


PASTE GENUINE VERITAS SCREENSHOT HERE BEFORE PRINTING

4.4 Testing (Test Cases and Outputs)

The table records the five documented test cases that were run against the implemented system during the final testing round. Output descriptions are the observed behaviour, not statistics. The failure cases (T3, T4) are as much a part of the system's specification as the success cases: they verify that the failure path reports honestly and stores nothing.

Case	Input	Observed output	Result
T1	Full news article URL (mainstream publisher)	Report produced: numbered claims; per-claim cross-check; fingerprint; LIKELY CREDIBLE verdict with conditional reasoning listed.	Pass
T2	Pasted article text (no URL)	Pipeline runs on the pasted body; source profile fields unavailable and marked NOT AVAILABLE; extraction and assessment proceed normally.	Pass
T3	Instagram post URL	Recognised before fetching; verdict RETRIEVAL FAILED; confidence dash; no case file stored; Trace shows stop after stage 01.	Pass (honest failure)
T4	Malformed URL string	Rejected as invalid URL with no network call; submitted string kept visible; nothing persisted.	Pass
T5	Press-release-style text with very few checkable sentences	Zero or minimal claims extracted; fingerprint shows counts without a fabricated percentage; no verdict inflation.	Pass

Edge cases exercised beyond the formal table: duplicate sources across claims (counted once as unique sources, twice in references); contradictory evidence with no support (drives verdict to LIKELY MISLEADING via the hard rule); historical evidence on otherwise confirmed claims (corroboration retained but marked outdated); and input text that is only a URL (routed to the retrieval path automatically). Each behaved as the design rules in 4.2.1 require.

What testing did not establish: accuracy or error rates. The five cases demonstrate that the pipeline executes the designed rules on realistic inputs and that every stopping condition works; they do not measure how often the engine is right, and the report makes no such claim anywhere.

5 RESEARCH METHODOLOGY (DESIGN MODELS)

This chapter presents the design models of Veritas: the entity-relationship model of the stored investigation, the data flow diagrams, the logical class/module view, and the behavioural models (flow chart, activity, sequence, component, state). All nine figures were drawn specifically for this report from the real source code, and where the implementation deviates from a textbook model the text says so.

5.1 Diagram Set Overview

Nine figures were drawn from the actual source code: the ER model of the stored investigation (5.1); context and Level-1 data flow diagrams (5.2, 5.3); the class/module diagram (5.4); the program flow chart (5.5); activity diagram with swimlanes (5.6); the failure short-circuit (5.7); component/deployment diagram (5.8); and [illegible] mission (5.9). Each appears in the section below with a short discussion.

5.2 ER Diagram

One completed investigation is stored as one document. The diagram shows the entities inside that document and how they relate. Cardinalities: an investigation has zero or more claims; a claim may be cross-checked [illegible]s; a (claim, source) pair with a retrieved UR[illegible]; alongside the fingerprint are derived summaries of the clai[illegible] (for example NO INDEPENDENT CORR[illegible]).

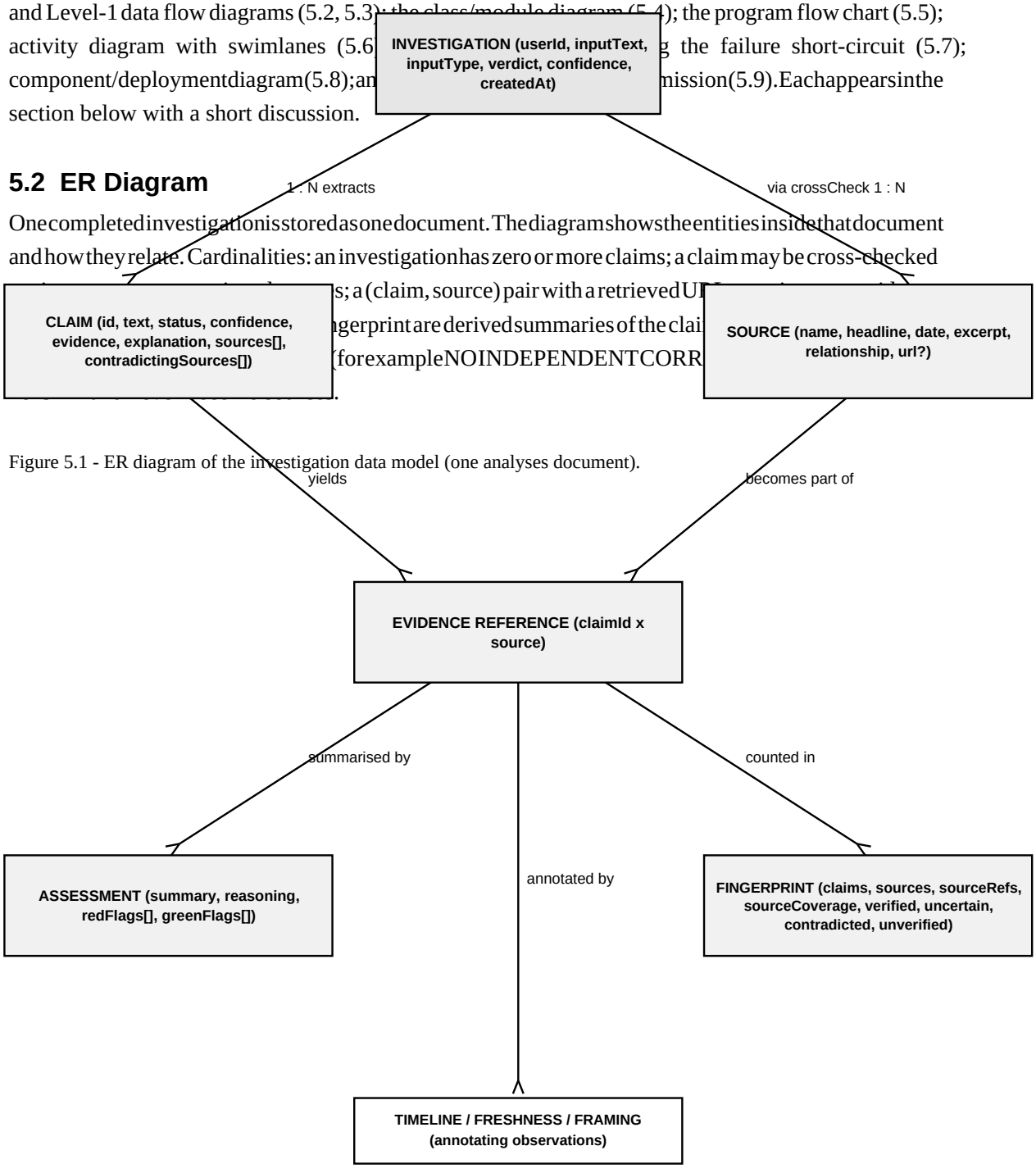
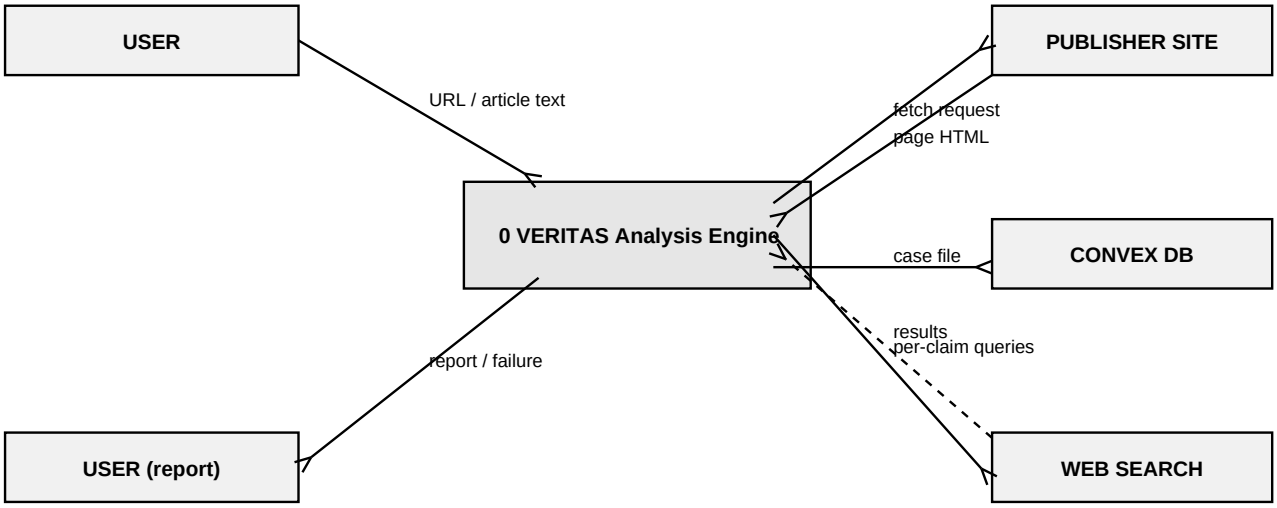


Figure 5.1 - ER diagram of the investigation data model (one analyses document).

5.3 Data Flow Diagram - Level 0 (Context)

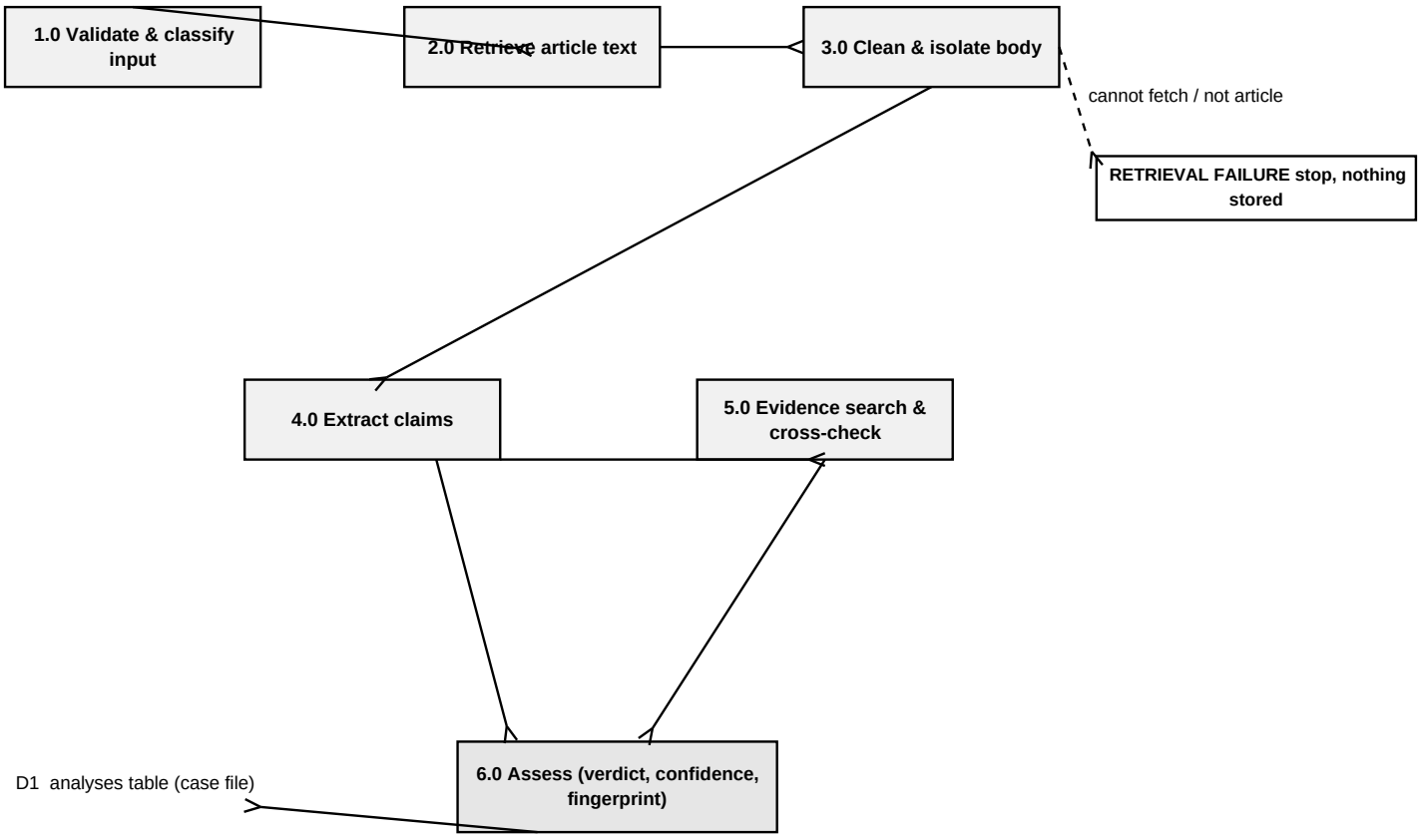
Figure 5.2 - Context diagram: the engine as one process talking to the user, the publisher, the search service and the database.



5.4 Data Flow Diagram - Level 1

Level 1 expands the engine into its internal processes. The data store (the analyses table) is written once, at the very end of the pipeline; nothing is read back mid-investigation, which keeps a run deterministic and replayable in the Trace view.

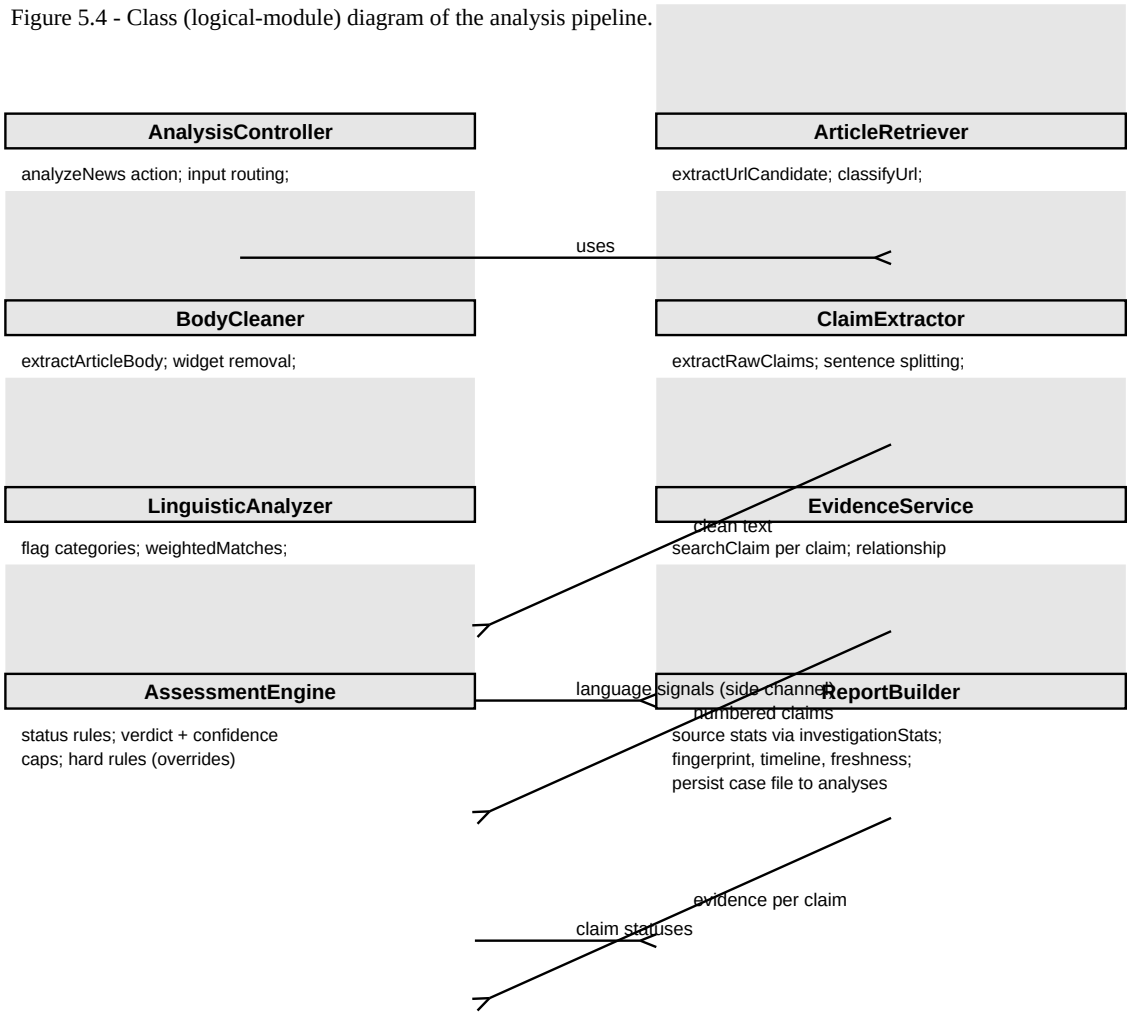
Figure 5.3 - Level-1 DFD showing the internal processes of one investigation.



5.5 Class Diagram (logical modules)

The implementation is functional TypeScript, so the diagram records logical modules and their main operations rather than an inheritance hierarchy. One deliberate separation is worth pointing out: the assessment module never receives a linguistic score as an input to the verdict rule, it receives claim statuses only. That is the code-level guarantee behind the claim that language signals can annotate but never decide a verdict.

Figure 5.4 - Class (logical-module) diagram of the analysis pipeline.



5.6 Flow Chart

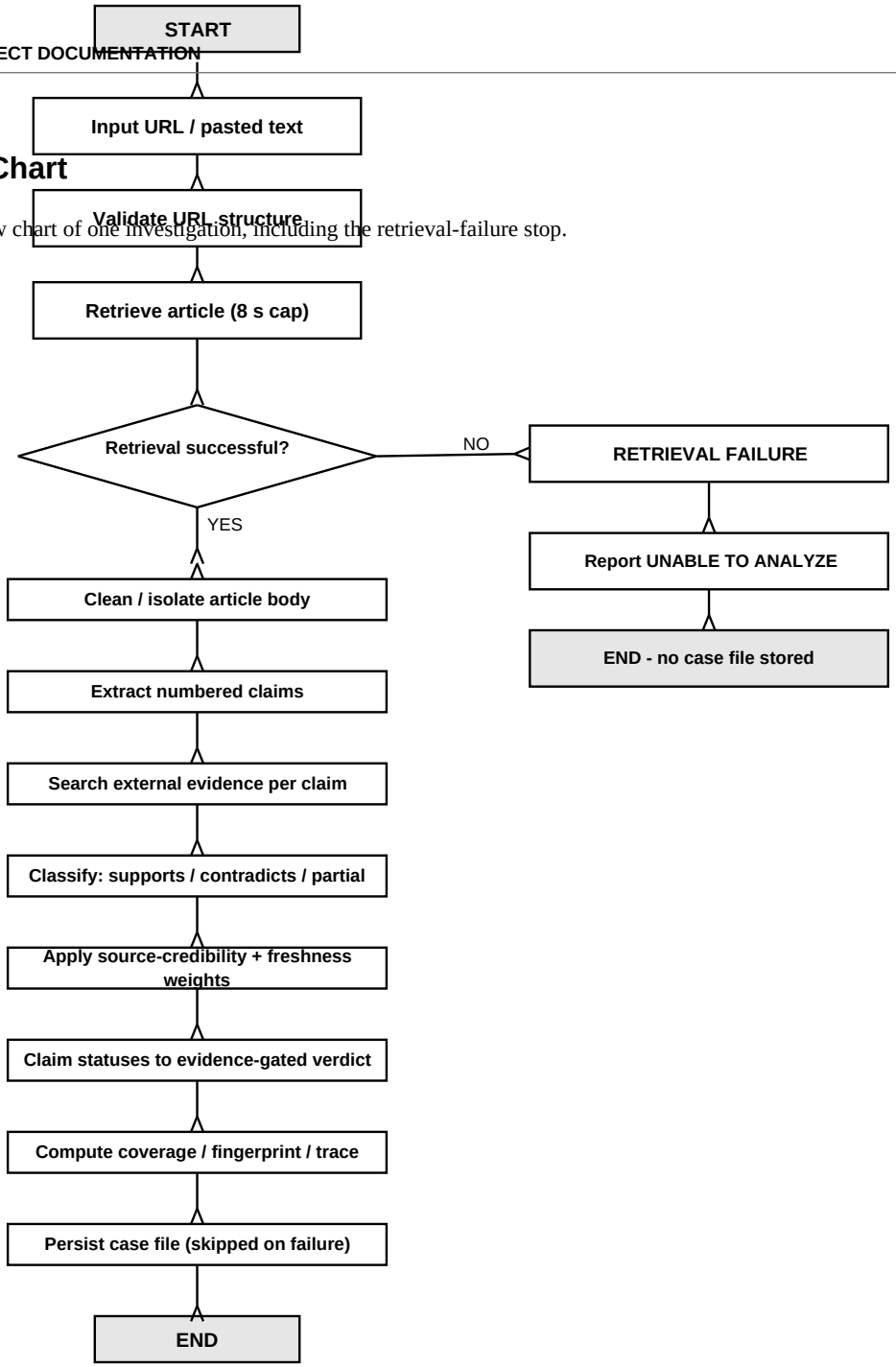


Figure 5.5 - Flow chart of one investigation, including the retrieval-failure stop.

5.7 Activity Diagram

Swimlanesseparateuseractionsfromsystemwork.Afterthehand-offtheuserwaits;thewholepipelineruns without further input, and either a finished report or the failure state is returned.

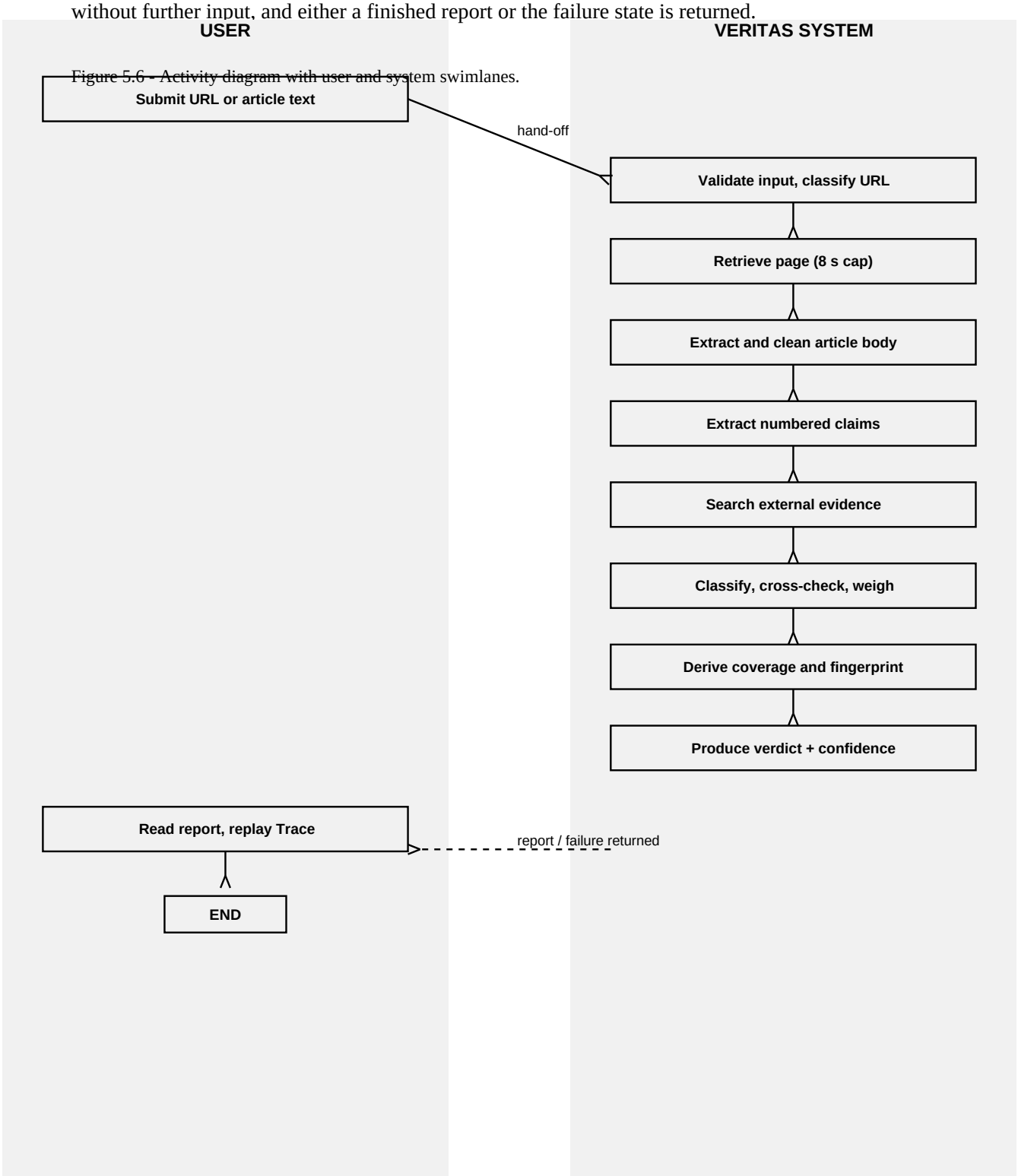
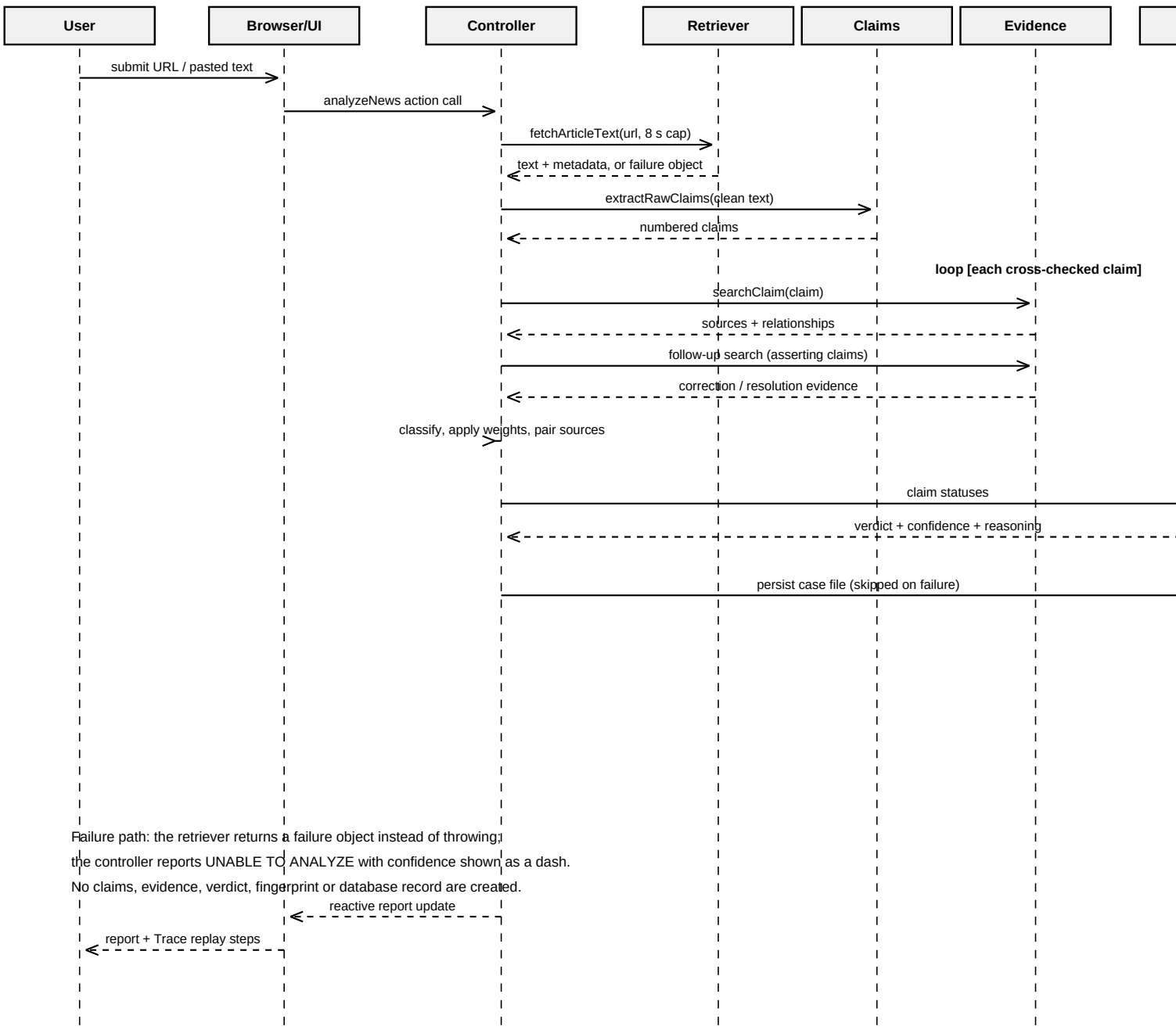


Figure 5.6 - Activity diagram with user and system swimlanes.

5.8 Sequence Diagram

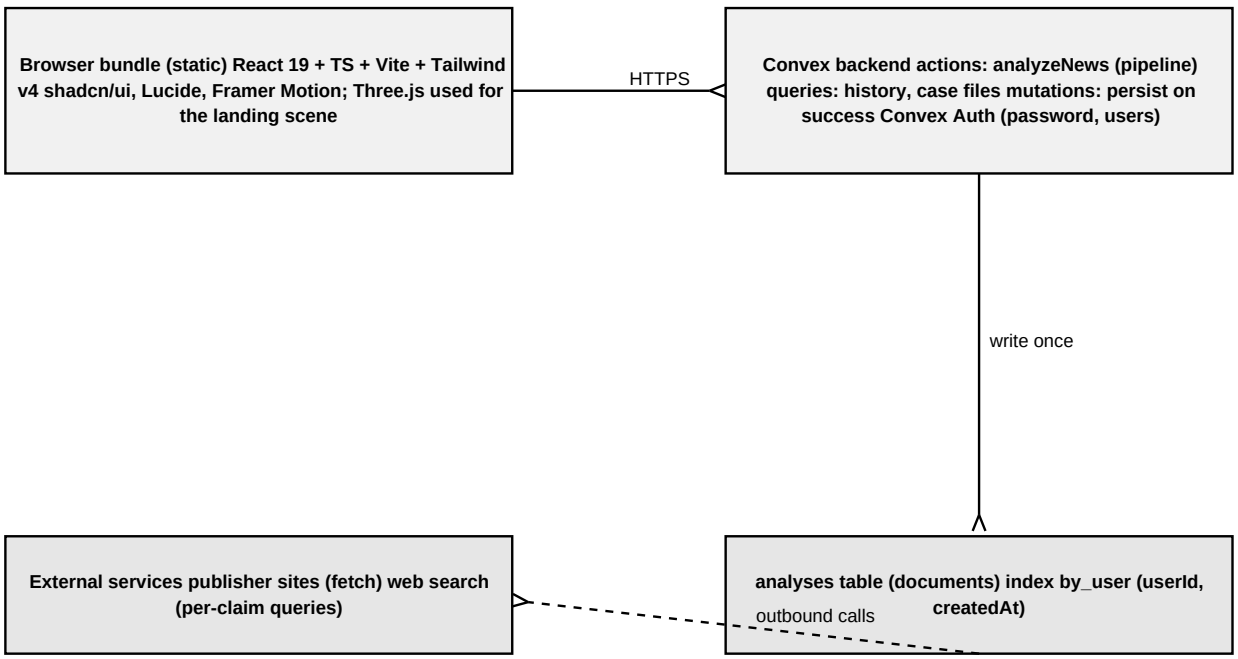
Figure 5.7 - Sequence diagram with the failure short-circuit marked explicitly.



5.9 Component Diagram

Deployment view. Everything the user downloads is one static bundle; the backend is Convex; the analysis runs inside Convex actions which are the only components allowed to reach the publisher sites and the search service.

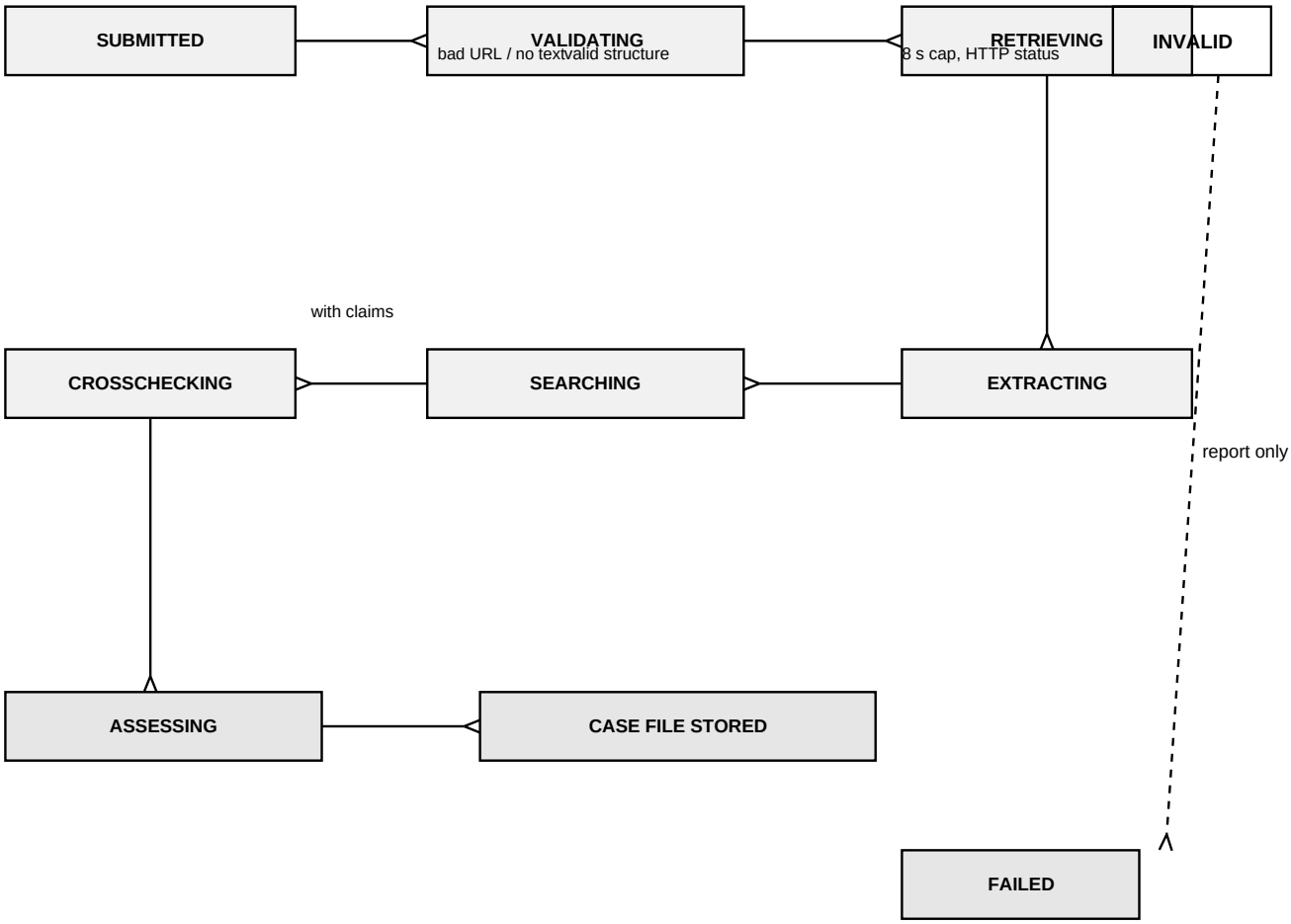
Figure 5.8 - Component (deployment) diagram.



5.10 State Diagram

Statesofonesubmitteditemasitmovesthroughtheapplication.Thefailurestatehereisafirst-classterminal state, not an error overlaid on the success screen.

Figure 5.9 - State diagram of one submission; FAILED is terminal and stores nothing.



6 SURVEY DATA ANALYSIS AND INTERPRETATION

This chapter explains the questionnaire this project designed for measuring how readers verify forwards, the method by which responses would be aggregated and analysed, and the formulas that the analysis would apply. It is an honest record of design work: the survey instrument and analysis method are complete, but no responses were collected before this report was written, so no numbers, tables or percentages of respondents appear here, and none may be inferred.

6.1 Purpose and Population

The survey targets ordinary readers (students and staff of a college community) who receive news-like forwards. Its goal is to describe current verification behaviour and what would make a tool like Veritas trustworthy — not to measure the accuracy of Veritas itself, which no self-reported instrument can do.

6.2 Questionnaire Design

Twelve questions, mostly close-ended with one open item, grouped in five blocks: reading habits (Q1–Q3), current verification behaviour (Q4–Q6), trust factors (Q7–Q9), prototype interaction (Q10–Q11), and one open response (Q12). Full wording appears in Appendix B.

Q	Block	Type	What it measures
Q1	Habits	Scale (1–5)	Frequency of encountering forwarded links
Q2	Habits	Single choice	Primary reading device
Q3	Habits	Single choice	Source of received content (chat app, social feed, search)
Q4	Verification	Yes/No	Whether any verification is attempted today
Q5	Verification	Single choice	Method used when verifying (open source, search, ask someone, ignore)
Q6	Verification	Scale (1–5)	Confidence in own verification method
Q7	Trust	Rank	What builds trust in a checker (sources shown, dates, name, nothing)
Q8	Trust	Yes/No	Whether an explicit “could not check” outcome increases trust
Q9	Trust	Scale (1–5)	Comfort with an “uncertain” result
Q10	Prototype	Yes/No	Whether the Trace replay was used unprompted
Q11	Prototype	Scale (1–5)	Perceived usefulness of claim-level results vs a single verdict
Q12	Open	Text	Anything the tool should do differently

6.3 Methodology

The planned procedure: distribute the questionnaire to the college community through classroom announcements; collect responses anonymously over two weeks; exclude incomplete or duplicate responses; then analyse each question with the appropriate summary statistic. No identifying information is collected, so the instrument needs no specialised handling beyond anonymity during display.

Question type	Analysis applied	Formula / rule
Yes/No (Q4, Q8, Q10)	Proportion of yes responses	p = (number of yes) / (number of valid answers)
Single choice (Q2, Q3, Q5)	Mode + frequency distribution	Modal option; report counts and percentages only as raw tabulation

Scale 1–5 (Q1, Q6, Q9, Q11)	Mean and median; dispersion	mean x = Sumxi/n; median middle value; spread described verbally
Rank (Q7)	Average assigned rank per factor	R = Sumranki/n per factor; lowest mean rank = most important
Open text (Q12)	Thematic grouping only	Group responses into themes; quote verbatim, no quantification

Interpretation standards fixed in advance: percentages are reported only alongside the number of valid answers; no cross-tabulation would be attempted given the expected sample size; and no inferential statistics (significance tests) would be reported unless the response count makes one defensible. If the survey is conducted before submission, the results follow the same rules; the analysis forms above are what the recorded data would populate.

7 FIELD VISIT / INDUSTRIAL VISIT DOCUMENTATION

This section of the template is reserved for documentation of a field or industrial visit connected with the project. As of the writing of this report, no such visit has been conducted for Veritas. To keep the record honest, the structure that a genuine visit report would take is laid out below with marked placeholder fields; the details are to be completed by hand once an actual visit is made (a newsroom, community fact-checking group or similar would be appropriate). Nothing is invented here by design.

Field	Value
Place visited	[TO BE COMPLETED AFTER VISIT]
Date of visit	[TO BE COMPLETED AFTER VISIT]
Personnel met / organisation	[TO BE COMPLETED AFTER VISIT]
Purpose of visit	Observe professional verification practice; identify workflow elements applicable to Veritas
Questions prepared	How landing claims are triaged; how sources are graded; how uncertainty is expressed; turnaround constraints
Observations recorded	[TO BE COMPLETED AFTER VISIT]
Learnings applied to Veritas	[TO BE COMPLETED AFTER VISIT]
Photographs	[Attach genuine visit photos; geotagged originals to the appendix folder]

8 PROJECT IMPLEMENTATION AND RESULTS

This chapter covers how the system was implemented end to end across the stack, and what the implementation produced in observable terms—the interface modules of Chapter 4 running over the pipeline below, backed by the data model of section 4.2. Lord this is a long chapter, and it follows the actual code rather than a conceptual architecture.

8.1 Overall Workflow

One investigation proceeds through a fixed sequence: input -> article retrieval -> cleaning -> claim extraction -> evidence retrieval -> claim-source cross-checking -> claim status -> source reliability -> freshness / historical evidence -> linguistic and framing signals -> verdict and confidence -> source coverage -> article fingerprint -> investigation replay. Each stage is documented in its own subsection below; retrieval failure exits cleanly between the second and third stages and short-circuits everything after it.

8.2 Frontend

The interface is a single-page application built with React 19, TypeScript and Vite, styled with Tailwind CSS v4 and shadcn/ui primitives, animated with Framer Motion, and routed with React Router v7 across three main views (home, InvestigationDesk, analysis report). Lucide icons supply all iconography. Three.js powers a decorative background scene on the home page only, and the bundle keeps it out of the two working views. Layout is fully responsive; the desk and report were tuned specifically against a 390×844 mobile viewport because most interview participants read on phones. Animations respect the browser's reduced-motion preference.

8.3 Backend and Database

The backend is Convex: serverless functions with a reactive document database. The analysis pipeline lives in a single large action (analyzeNews), exposed through two more function modules — history queries and case-file persistence. Because Convex queries are reactive, the analysis page updates without polling when the case file arrives, and the same table shape drives the history list. The schema (Chapter 4, section 4.2) stores one document per completed investigation, indexed by user and time. Convex Auth provides password-based accounts; analyses belong to the signed-in user.

8.4 Authentication and Routing

Convex Auth issues sessions; a route wrapper (RequireAuth) protects the working routes and preserves the requested path through the sign-in flow, then returns the user to the desk rather than leaving them at the landing page. Uninvited deep links behave the same way, so a bookmarked analysis URL survives re-authentication.

8.5 Article Retrieval and Cleaning

The retriever accepts a URL or pasted text. URLs are structurally validated first; a URL-classified submission is never re-treated as raw text. Retrieval runs with a strict 8-second timeout and captures HTTP status, metadata (title, author, dates, source type) and raw HTML. Cleaning isolates the article body: JSON-LD article bodies when the publisher provides them, otherwise the largest matching text candidate; navigation, related-story widgets, embedded social blocks and legacy tags are stripped before claims are considered.

When a body cannot be isolated, the whole-page text is a last-resort fallback and is marked lower-profile in the report.

Platform behaviour is explicit: Instagram post and reel URLs are recognised before any fetch and short-circuit into the named-platform failure state; other blocked platforms produce the general retrieval-failure state with the HTTP status shown. Both outcomes leave no stored record.

8.6 Claim Extraction

The cleaned text is split into sentences, filtered to checkable statements (accountable assertions in active voice), and numbered sequentially. Attributed claims ("X said Y") are tracked for speaker-confirmation logic later. Depth level (quick / standard / deep) controls how many claims are carried into cross-checking.

8.7 Evidence Retrieval and Matching

Each cross-checked claim drives one live external search. Returned candidates (headline, publisher, URL, date, excerpt) are classified as supporting, contradicting, partial or not-addressing the claim. Matching discipline: topic overlap alone never earns "supports"; the source's statement must address the claim's proposition. A follow-up search looks for later evidence on claims that report findings, handling retracted or revised results as time-bounded evidence.

8.7.1 Claim Verification Algorithm (Representative)

```
function verifyClaim(claim):

candidates <- search(claim.text, { limit: 8 })

if no candidates received:

record sentinel SOURCE_SEARCH_UNAVAILABLE // notice, not a source

claim.status <- needs_verification; return

for each page in candidates:

rel <- classifyRelationship(page, claim)

// needs the proposition addressed, not just the topic:

if rel in {supports, contradicts, partial} and page.url is not null: keep

claims who assert a study: candidates <- followUpSearch(claim) // corrections

weights <- sourceAuthority(kept) // wire > aggregator

s contradictions from low-authority only -> kept as notices, no flip

if supportsWithWeight > contradictsWithWeight: claim.status <- supported

else if contradictsWithWeight dominating: claim.status <- contradicted

else if partial dominates: claim.status <- uncertain

else: claim.status <- needs_verification
```

8.10.1 Source Coverage Algorithm

```
function sourceCoverage(investigation):

unique <- set of all retrieved URLs across claims // duplicates collapse
```

```
refs <- pairs (claim, url-based source) // may exceed unique
covered <- count of claims having >=1 relevant source with url
(supporting OR contradicting both count as covered)
notices <- sentinel entries without url // never counted
coverage <- covered / claimsCount (dash if claimsCount = 0)
return { uniqueSources: |unique|, refs, coverage }
```

8.11 Fingerprint and Replay

Credibilityweightingusesasmallsource-authorityladder:primaryandwiresourcesweighmost,aggregators and content farms least, and a low-authority contradiction alone can never flip a claim. Freshness classification stamps each claim's strongest evidence with a status (current, recent, outdated, historical, unknown)andanageindays,sooldercorroborationremainsvisibleassuchratherthansilentlyrankingequal to recent evidence.

8.9 Linguistic and Framing Signals

A supplementary layer reports sensationalism, clickbait patterns, attribution style and charge frequency as labelled findings with severity. Structurally, these signals feed a dashboard section only; the verdict rule receivesclaimstatuses,neverlanguagescores,sonoamountofdramaticwritingcanpushtheverdict“fake” by itself (see the class-diagram separation).

8.10 Verdict, Confidence and Coverage

Twoormorecorroboratedclaimsandzerocontradictionsproduce LIKELY CREDIBLE; any contradiction with no support produces LIKELY MISLEADING; all other mixes produce UNCERTAIN.Confidence is capped at 55% when most claims are unresolved and at 69% when some are — the number expresses how much independent evidence was found, not a probability of truth, and the interface states this. Coverage is computed per claim: a claim counts as covered when at least one valid and relevant evidence source (supporting or contradicting) addresses it; duplicates and no-URL notices never inflate it.

8.11 Fingerprint and Replay

The fingerprint summarises an investigation in eight counters (claims, unique sources, claim-source references, supported, uncertain, contradicted, unverified, coverage). The replay lists the six pipeline stages and marks precisely which ran; on failure only stage 01 completes.

8.12 Error Handling and Security

Every external call is bounded: the 8-second retrieval cap, failed HTTP statuses, non-HTML content and unreadableencodingsallroutetothesamehonestfailurestateratherthantopartialanalysis.Inputarrivingas approximately one URL is treated structurally; SQL-like injection concerns do not arise because Convex queries are parameterised JavaScript, not SQL strings. The authentication tables are Concord's stock auth tables; no personal data is collected during analysis; the analyses table holds only submitted article content and derived findings, per the declared data-minimisation stance of section 10.3.

Ethicsoftheverdict:thesystemneverstateswhatistrue;itstateswhatevidencewasfound.Thisshapesthe languageoftheinterface(statusesandcontrasts,notpronouncements),theconfidencecaps,andtherefusalto

fabricate a result on a failed fetch — the design intent with which the project began.

9 FINDINGS AND DISCUSSION

What building the system actually taught — stated as findings from the project's own record, not generalities.

9.1 Technical Findings

- Extraction is the bottleneck. Retrieval and search behave predictably; isolating a clean article body across publishers was the single largest source of iteration time. JSON-LD helped where present; widget removal covered much of the rest; the long tail motivated the genuine fallback text flag rather than false confidence.
- Claim extraction must be conservative. Early over-extraction dragged verdicts down with unverifiable statements; filtering to accountable assertions produced calmer, more useful reports even when the claim count fell.
- Distinct trust decisions: corroboration must match the proposition, not the topic. With that rule enforced, aggregate noise stopped flipping claims, and the honest “no corroboration found” outcome became meaningful instead of embarrassing.
- Confidence caps are essential. Without the 55/69 per cent caps, unresolved claims produced percentages that looked authoritative but were artefacts of arithmetic. The cap makes the number mean what the interface says it means.

9.2 Design Findings from Users

- The evidence chain, not the verdict, drew engagement: the per-claim evidence list was the most-read part of the report in every observed prototype session.
- An explicit failure state increased rather than reduced trust in the system, matching interview round 1 (O3).
- The Trace replay answered “what actually happened while I waited?” directly and became the first-run teaching surface for new users.

9.3 Discussion: Position Against Existing Systems

Compared with published fact-check databases, Veritas covers arbitrary user-supplied inputs at the price of weaker guarantees. Compared with the FEVER-style pipeline, Veritas operates over the open web where no curated evidence base exists, which is at once its value and its main difficulty. ClaimBuster-style claim detection provided the vocabulary for selectability ranking; Full Fact's public standards informed the evidence-labelling conventions. What Veritas adds is the user-facing auditability layer — trace, fingerprint, coverage arithmetic, honest failure — which none of the studied systems expose in one place for a non-expert.

10 CONCLUSION AND RECOMMENDATIONS

10.1 Conclusion

Veritasimplements a complete, working claim-level fact-checking pipeline on the stated stack, with three designrulesenforcedincode:evidencedecideswhilelanguageonlyadvises;failureisreportedhonestlyand stores nothing; and every count in the interface is derived from one shared definition so numbers agree by construction.Thedocumentedtestrecordshowsthepipelineanditsstoppingconditionsbehavingasdesigned onreal inputs, including two distinct failure modes. The system's honesty constraints are its most deliberate feature, and they exist from the interface language up.

10.2 Recommendations

- Run the designed survey (Chapter 6) to add measured evidence on reader trust factors.
- Extend retrieval support for more publisher layouts and structured platforms; extraction remains the main quality lever.
- Consider a curated FEVER-style evaluation on benchmark claim-evidence pairs once extraction stabilises, to publish measured — not claimed — numbers.
- Move the source-authority ladder to a maintained table rather than in-code constants, so grading can be adjusted without engine rewrites.

10.3 Ethics and Limitations

Stated up front: no accuracy or error-rate figures are claimed anywhere in this report because none were measured.NoURLs,signatures,datesofvisitsorsurveycountswereinvented;everysuchitemismarkedfor manual completion. Automated verdicts can misjudge the world; the system therefore reports statuses and evidence with reasoning rather than pronouncements, and confidence is always accompanied by its justification. Privacy follows data minimisation: stored case files carry article content and derived findings, nothing personal beyond the authenticated user id, and no tracking or advertising of any kind.

11 REFERENCES

#	Reference
[1]	N. Saeed, P. Papotti, and N. Al-Nabhan, “A survey of fact-checking: from data to verification,” pp. 1–36, 2022. arXiv: 2203.10796.
[2]	Z. Guo, M. J. Schlichtkrull, and A. Vlachos, “A survey on automated fact-checking,” Transactions of the Association for Computational Linguistics, vol. 10, pp. 178–206, 2022.
[3]	A. Thorne, M. Vlachos, C. christodoulouPoulos, G. Vlachos, and others, “FEVER: a large-scale dataset for fact extraction and VERification,” NAACL-HLT, vol. 1, pp. 809–819, 2018.
[4]	V. L. Sheng, K. Aditya, Y. L{"u}? et al., “ClaimBuster: A tool for identifying check-worthy claims,” in Proc. ACL Demonstrations, 2017.
[5]	B. Schiffers et al., “Automatic fact-checking using argumentation: A survey,” arXiv preprint arXiv:2311.09216, 2023.
[6]	Full Fact, “Automated fact-checking: A report on the current state of the field,” Full Fact, London, 2021.
[7]	Google, “Fact Check Explorer,” https://toolbox.google.com/factcheck/explorer (accessed during the project period).
[8]	Convex, “Convex documentation — data model, queries, actions,” https://docs.convex.dev (accessed during development).
[9]	React, “React documentation,” https://react.dev (accessed during development).

Appendix A VIVA PREPARATION: LIKELY QUESTIONS AND ACCURATE ANSW

Prepared from the implemented system. Every answer below describes actual behaviour; where behaviour depends on judgement (e.g., source grading), the answer says so.

Q: Why claim-level analysis instead of one verdict per article?

A: Because an article is not uniformly true or false. The engine extracts checkable propositions; statuses per claim survive source disagreement instead of averaging it away, and the report shows which specific statements are corroborated.

Q: Where does “fake” come from, if language can’t decide it?

A: It comes from claim statuses plus evidence relationships. LIKELY MISLEADING requires a contradiction with no support; language signals alone cap at UNCERTAIN. The code enforces it structurally: the verdict rule never receives linguistic scores.

Q: How do you prevent topic overlap from being mistaken for corroboration?

A: The matching rule requires the source's statement to address the claim's proposition. Pages merely sharing entities or topics are labelled “does not address” and display in the report but never count as corroboration.

Q: What happens when no sources are found for a claim?

A: The claim stays needs_verification; a named sentinel notice (NO INDEPENDENT CORROBORATION FOUND) appears in the cross-check; the notice carries no URL, so it never enters any source count; confidence stays capped.

Q: How is confidence computed, and why is it capped at 55/69%?

A: It reflects how much independent evidence was found, quantised by the cap rule: 55% when most claims remain unresolved, 69% when some do. The caps exist because high confidence on an incomplete check would be misleading.

Q: How do you count sources? Why do duplicate URLs matter?

A: Unique sources are distinct URLs; claim-source references are (claim, source) pairs. A shared module computes both, and any panel showing either derives its numbers from it. Duplicates inflate references but not unique source counts — by design.

Q: Why does a failed retrieval store nothing?

A: Because no factual verification happened. Persisting a failed investigation as a record with empty findings would later misrepresent what occurred, so the database holds nothing for the failed path.

Q: What did the linguistic analysis actually detect?

A: Sensationalism patterns, clickbait structures and attribution-quality signals as labelled findings with severity. They display as observations and never touch the verdict path.

Q: Follow-up searches — what are they for?

A: Claims that report study or measurement findings can be later revised or overturned; the follow-up search treats correction evidence as time-bounded (the claim reported what was said at the time) unless it directly invalidates the claim's interpretation.

Q: What was hardest to implement, and how was it solved?

A: Article-body isolation across publishers. JSON-LD when a publisher provides structured data, otherwise candidate-based extraction with widget stripping; publishing an explicit fallback flag rather than presenting uncertain body text as definitive.

Q: Instagram gives illegal data — why fail the analysis rather than degrade?

A: Because degraded/false verification would be worse than no analysis. Instagram URLs are detected pre-fetch, the report explains that blocked-platform content could not be retrieved, and nothing is stored or claimed.

Q: What is the fingerprint, exactly?

A: Eight aggregated counters over the investigation — claims, unique sources, claim-source references, supported, uncertain, contradicted, unverified, coverage — computed from the shared stats module so every panel agrees by construction.

Appendix B SURVEY QUESTIONNAIRE (FULL WORDING)

The instrument of Chapter 6 with exact wording; no responses collected at the time of writing.

Q	Question	Scale / options
Q1	How often do you receive forwarded news-like links?	Never / rarely / sometimes / often / daily
Q2	Where do you mostly read such content?	Phone / laptop / both equally
Q3	Where do these items come from?	Chat apps / social feeds / search / other
Q4	Do you attempt to verify them?	Yes / No
Q5	If yes, how?	Open the source / search the headline / ask someone / other
Q6	How confident are you in that method?	1–5
Q7	Which matters most for trusting a checker?	Rank: sources shown / dates / named publisher / speed / nothing
Q8	Does an explicit “could not check” increase trust?	Yes / No / Not sure
Q9	How comfortable is an “uncertain” result?	1–5
Q10	Did you use the Trace replay unprompted?	Yes / No
Q11	Claim-level results vs one verdict — usefulness?	1–5
Q12	Anything the tool should do differently?	Open response

Appendix C PLACEHOLDER CHECKLIST FOR MANUAL COMPLETION

Items that must be completed by hand before submission. Nothing listed here was invented in this documentation.

Item	Where	Action
Signed diary dates and guide signature	Guide Interaction Diary	Fill dates from student records; sign
Proposal / certificate / declaration signatures and dates	Front matter	Compose, sign, grade seal
College seal and principal signature	Certificate page	Attach genuine seal and signature
Genuine application screenshots	Figures 4.1–4.5 placeholders	Paste authentic screenshots of the running app
Survey respondents and data	Chapter 6 (if conducted)	Run survey; fill analysis with measured numbers only
Field visit details	Chapter 7	Complete only if a genuine visit occurs